Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Greedy Algorithm Design Technique

- A greedy algorithm is any algorithm that follows the problem solving heuristic of making the locally optimal choice at each stage with the intent of finding a global optimum.
- We can make choice that seems best at the moment and then solve the sub problems that arise later.
- The choice made by a greedy algorithm may depend on choices made so far, but not on future choices or all the solutions to the sub problem.
- It iteratively makes one greedy choice after another, reducing each given problem into a smaller one.
- A greedy algorithm never reconsiders it's choices.
- A greedy strategy may not always produce an optimal solution.

e.g. Kruskals MST
Prims MST
Dijkstra's SPT

- Greedy algorithm decides minimum number of coins to give while making change.
- Available coins : 1, 2, 5, 10, 20, 50

change = 36

36 - 50 = -14 ✗
36 - 20 = 16 (20)
16 - 10 = 6 (10)
6 - 5 = 1 (5)
1 - 2 = -1 ✗
1 - 1 = 0 (1)

Recursion

- Function calling itself is called as recursive function
- For each function call, stack frame is created on the stack.
- Thus it needs more space as well as more time for execution.
- However recursive functions are easy to program.
- Typical divide and conquer problems are solved using recursion. back tracking
- For recursive functions two things are must
    1. Recursive call (explain process in terms of itself
    2. Terminating or base condition (where to stop)

Recursion - Fibonacci series (n^{th} term)

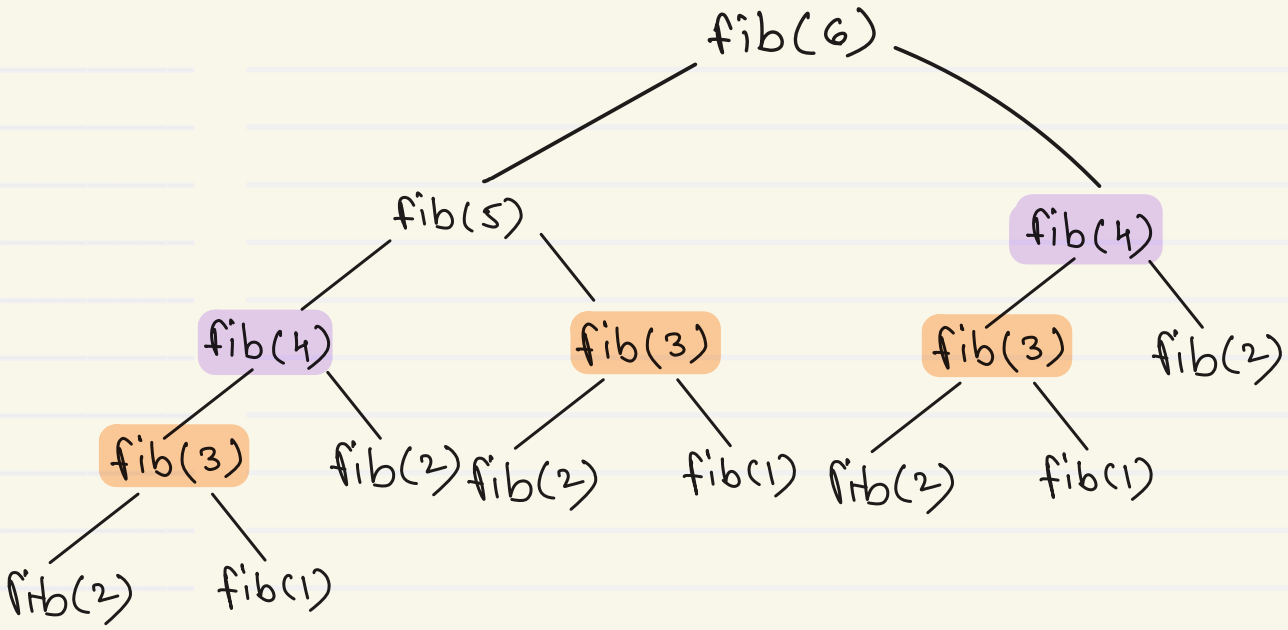
- **Recursive formula**

$$T_n = T_{n-1} + T_{n-2}$$

- **Terminating condition**

$$T_1 = T_2 = 1$$

- **Overlapping sub problem**



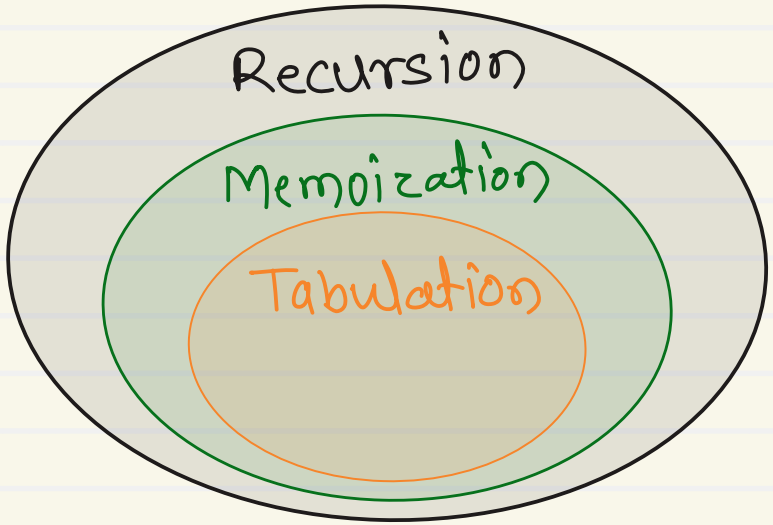

```
int fib(int n) {
    if(n==1 || n==2)
        return 1;
    return fib(n-1) + fib(n-2);
}
```

$$T(n) = O(2^n)$$ ← exponential growth

Dynamic programming

Dynamic Programming

- Memoization
  - Recursion + 1D or 2D Arrays
  - (Top Down Approach)
- Tabulation
  - Loops + 1D or 2D Arrays
  - (Bottom Up Approach)



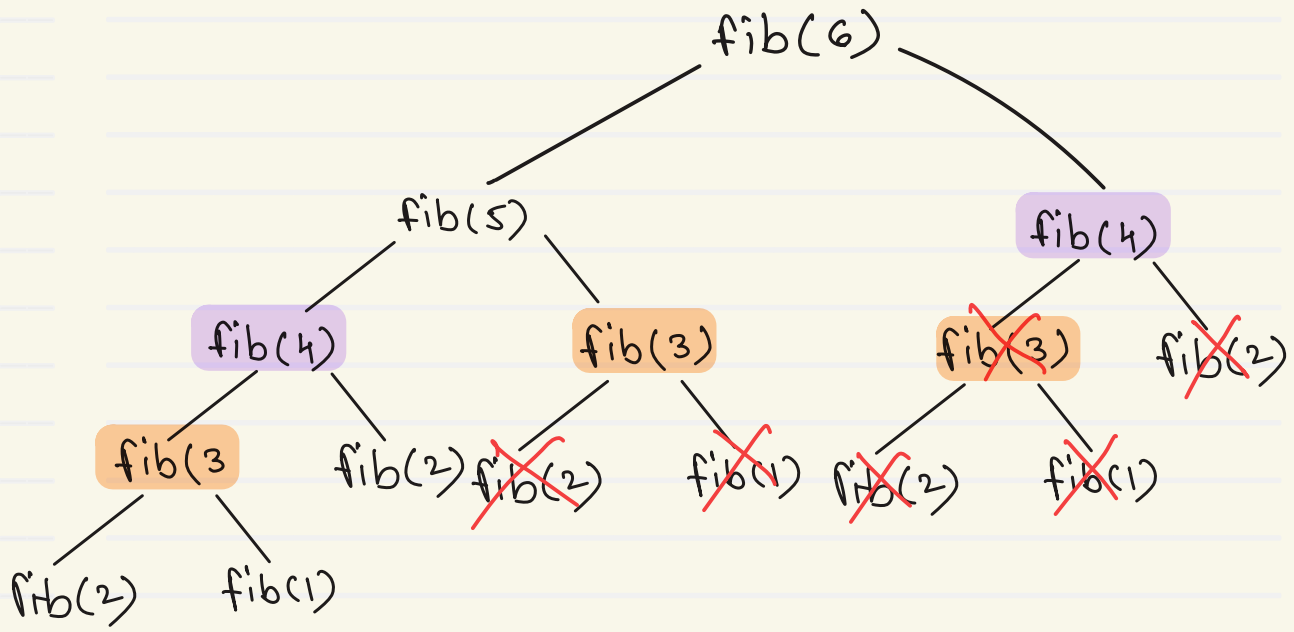
Dynamic programming - Memoization

- It's based on the Latin word memorandum meaning "to be remembered"
- Memoization is a technique used in computing to speed up programs.
- This is accomplished by memorizing the calculation results of processed input such as the results of function calls.
- If the same input or a function call with the same parameters is used, the previously stored results can be used again and unnecessary calculation are avoided.
- Need to rewrite recursive algorithms. Using simple arrays or map/dictonary.
- Memoization uses top down approach.

Memoization - Fibonacci series

```
int dp[] = new int[n+1];

int fib(int n){
    if(dp[n] != 0)
        return dp[n];
    if(n==1 || n==2){
        dp[n] = 1;
        return 1;
    }
    dp[n] = fib(n-1) + fib(n-2);
    return dp[n];
}
```

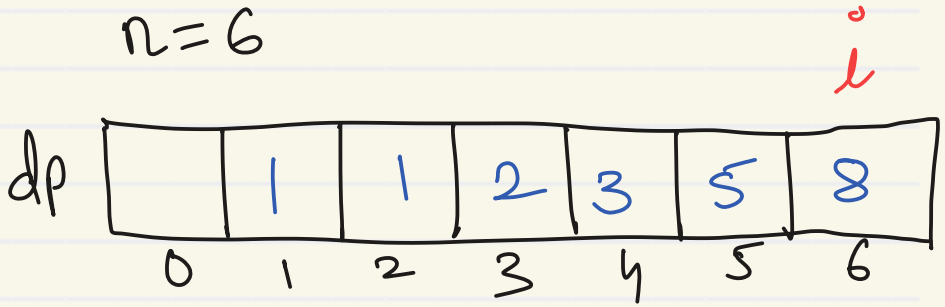


Dynamic programming - Tabulation

- Dynamic programming is another optimization over recursion.
- Typical DP problem give choices (to select from) and ask for optimal result (maximum or minimum).
- Technically it can be used for the problems having two properties
    1. Overlapping sub problems
        - To solve problem, we need to solve it's sub problems multiple times.
    2. Optimal sub structure
        - Optimal solution of problem can be obtained using optimal solutions of its sub problems.
- DP solution is bottom up approach.
- DP uses 1D or 2D array to save state.
- DP algorithms solve the sub problem in a iteration and improves upon it in subsequent iterations.

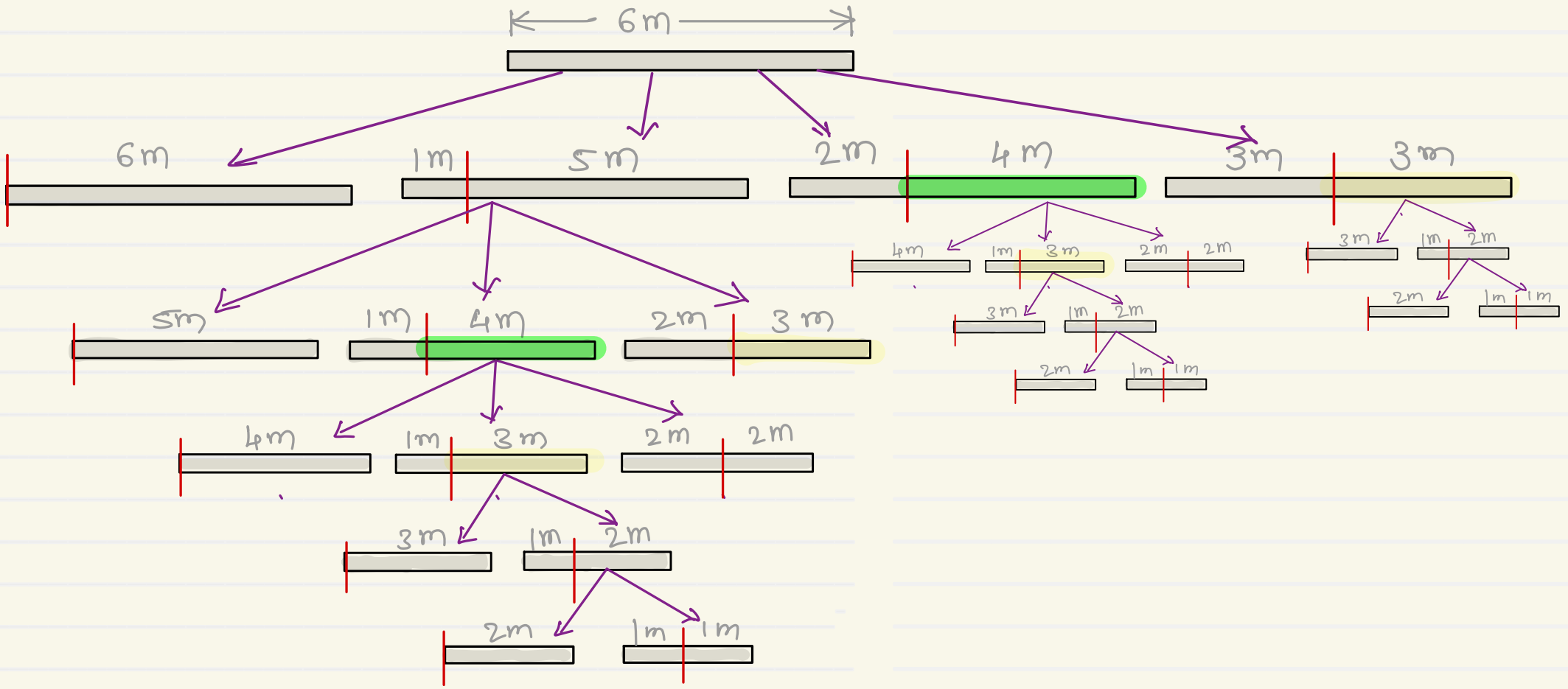
Tabulation - Fibonacci series

```
int dpfib(int n) {
    int dp[] = new int[n+1];
    dp[1] = dp[2] = 1;
    for(int i=3; i<=n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```



Dynamic programming - Rod cutting problem

- Cut the rod of given price so that maximum profit can be achieved.



Length	Price
1	1
2	5
3	8
4	9
5	10
6	14
7	17
8	20
9	24
10	30

Dynamic programming - Rod cutting problem

- Cut the rod of given price so that maximum profit can be achieved.

rod(1) = 1

rod(2) = 5
- rod(2) = 5
- rod(1) + rod(1) = 2

rod(3) = 8
- rod(3) = 8
- rod(1) + rod(2) = 6

rod(4) = 10
- rod(4) = 9
- rod(1) + rod(3) = 9
- rod(2) + rod(2) = 10

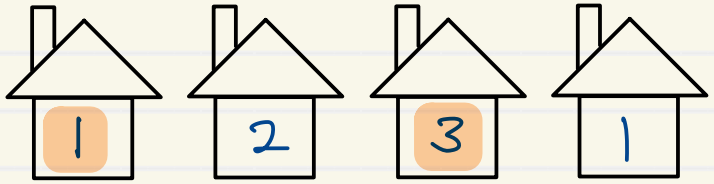
rod(5) = 13
- rod(5) = 10
- rod(1) + rod(4) = 11
- rod(2) + rod(3) = 13

rod(6) = 16
- rod(6) = 14
- rod(1) + rod(5) = 14
- rod(2) + rod(4) = 15
- rod(3) + rod(3) = 16

Length	Price
1	1
2	5
3	8
4	9
5	10
6	14
7	17
8	20
9	24
10	30

Dynamic programming - House robbing problem

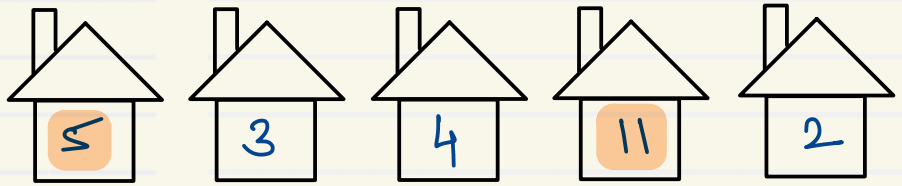
- Rob the houses to get maximum loot but no adjacent houses can be robbed.



1 + 3 = 4

2 + 1 = 3

1 + 1 = 2



5 + 4 + 2 = 11

3 + 11 = 14

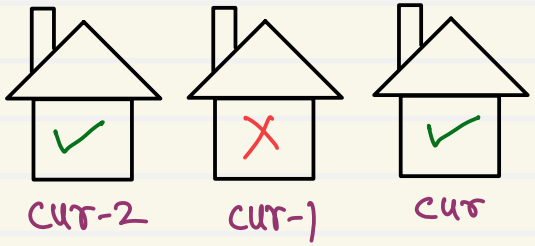
5 + 11 = 16

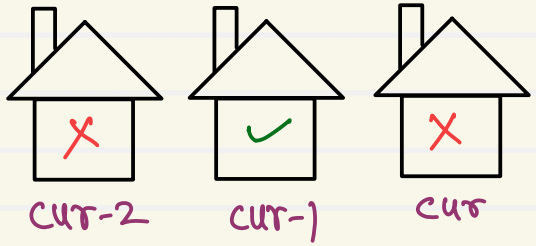
5 + 2 = 7

3 + 2 = 5

4 + 2 = 6

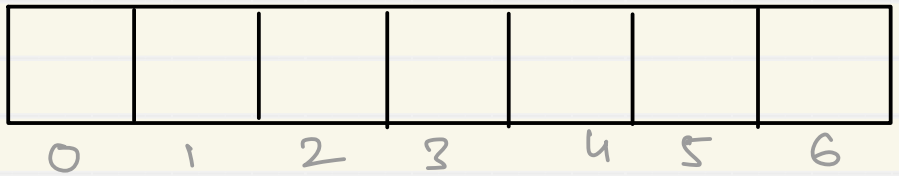
4 + 5 = 9

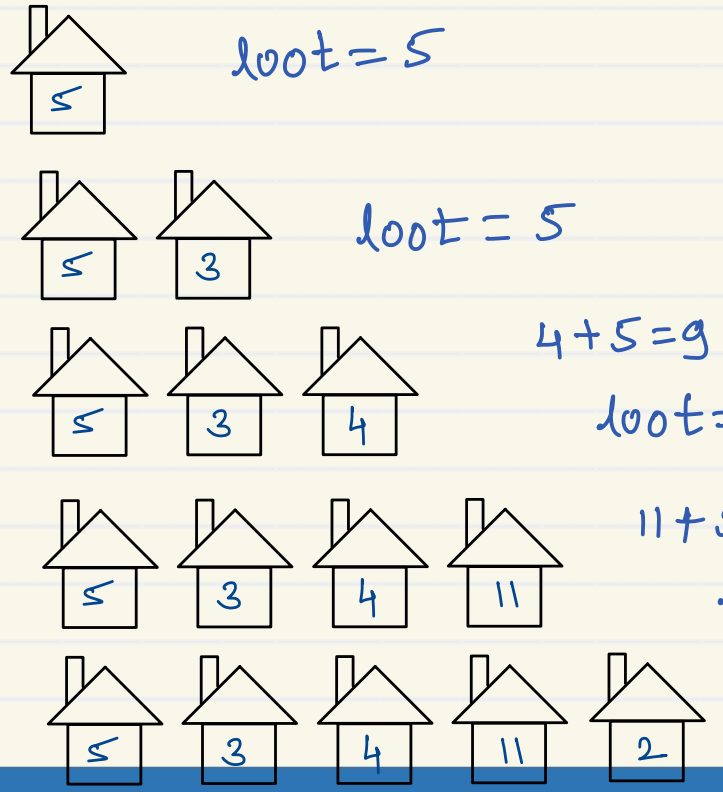


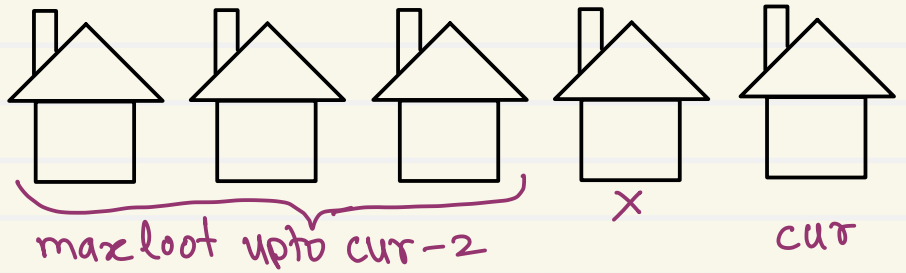


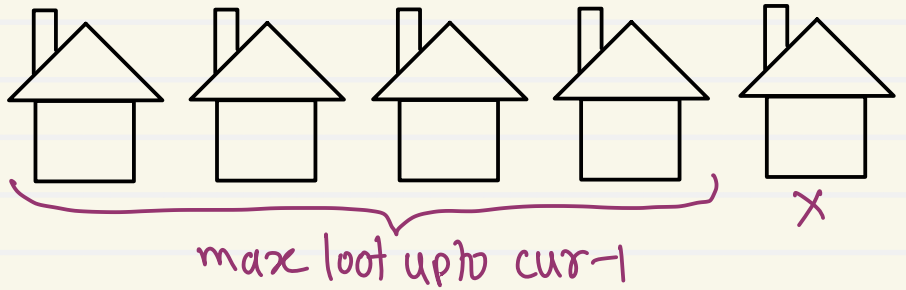
Dynamic programming - House robbing problem

- Rob the houses to get maximum loot but no adjacent houses can be robbed.





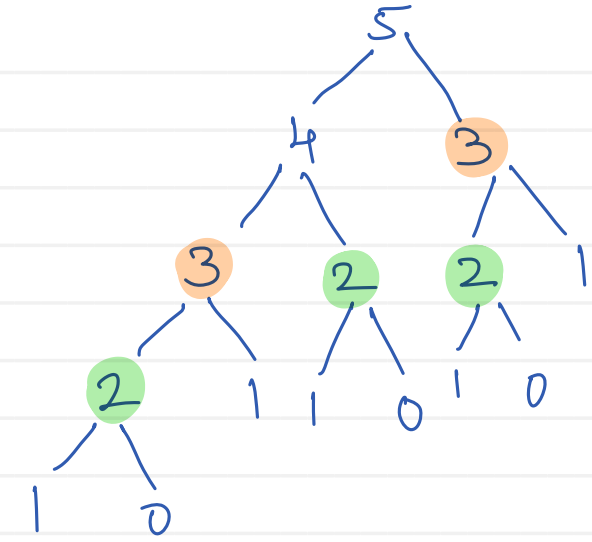




loot = MAX(curr + maxloot[cur-2], maxloot[cur-1])

Climbing Stairs

- You are climbing a staircase. It takes n steps to reach the top.
- Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?
- Example 1:
  - Input: n = 2
  - Output: 2
  - Explanation: There are two ways to climb to the top.
    - 1. 1 step + 1 step
    - 2. 2 steps
- Example 2:
  - Input: n = 3
  - Output: 3
  - Explanation: There are three ways to climb to the top.
    - 1. 1 step + 1 step + 1 step
    - 2. 1 step + 2 steps
    - 3. 2 steps + 1 step



n = 0 → 0
= 1 → 1
= 2 → 2
= 3 → 1+2 = 3
= 4 → 3+2 = 5
= 5 → 5+3 = 8